

Relatório do Trabalho 1 de Criptografia

2025/02

Gustavo do Prado Silva - GRR20203942¹, Nicolas André Rizzardi - GRR20206152¹

¹ Departamento de Informática – Universidade Federal do Paraná (UFPR)
Curitiba – PR – Brasil

Resumo. *Este relatório descreve a implementação e funcionamento da cifra criada para o primeiro trabalho da disciplina de Criptografia. Foram criadas e implementadas um cifra de substituição baseada em sementes aleatórias e um cifra de transposição baseada em Columnar Transposition, podendo elas serem aplicadas em conjunto ou separadas baseado em flags passadas ao programa. Essas cifras foram executadas juntamente com o AES em um conjunto predefinido de textos claros e seus tempos de execução foram comparados.*

1. Introdução

O objetivo desse trabalho é "criar" uma cifra nova utilizando as cifras aprendidas em sala de aula, sendo uma de substituição e uma de transposição. Depois, vamos comparar o tempo de execução da cifra criada com o tempo de execução do AES para os mesmos textos claros e gerar gráficos comparando ambos.

2. Cifras

2.1. Cifra de Substituição

2.1.1. Ideia

Usar a chave dada como *seed* e a partir disso, gerar caracteres aleatórios que serão combinados com os caracteres do texto para gerar o texto cifrado final. Como o XOR é uma operação reversível, será utilizada a mesma função tanto para cifrar quanto para decifrar. Para não termos problemas ao ler caracteres possivelmente inválidos gerados, vamos ler e escrever os arquivos como bytes, de 4096 em 4096.

2.1.2. Passos

Para criptografar:

1. Iniciar a *seed* da biblioteca *random* com a chave dada pelo usuário
2. Para cada byte lido do texto claro:
 - (a) Sortear um número aleatório entre 0 e 255 (maior byte possível)
 - (b) Fazer um XOR entre o byte lido e o byte aleatório sorteado
 - (c) Adicionar o novo byte a saída

Para descriptografar:

1. Iniciar a *seed* da biblioteca *random* com a chave dada pelo usuário
2. Para cada byte lido do texto criptografado:
 - (a) Sortear um número aleatório entre 0 e 255 (maior byte possível)
 - (b) Fazer um XOR entre o byte lido e o byte aleatório sorteado
 - (c) Adicionar o novo byte a saída

2.2. Cifra de Transposição

2.2.1. Ideia

Usar Transposição Colunar (Columnar Transposition) por duas iterações para cifrar o texto usando transposição. Vamos inicialmente criar uma matriz de transposição, depois reordenaremos ela de acordo com a ordem alfabética da chave, assim o texto é escrito em linhas, mas lido em colunas reordenadas.

T	O	M	A	T	O
5	3	2	1	6	4
T	H	E	T	O	M
A	T	O	I	S	A
P	L	A	N	T	I
N	T	H	E	N	I
G	H	T	S	H	A
D	E	F	A	M	I
L	Y	X	X	X	X

Figura 1. Exemplo de matriz de transposição colunar

2.2.2. Passos

Para criptografar:

1. Leia o texto claro da entrada
2. Por duas rodadas, faça:
 - (a) Pegue a chave da rodada
 - (b) Crie a matriz de transposição do texto junto com a chave
 - (c) Complete a última linha da matriz com caracteres
 - (d) Reordene o texto baseado na ordem alfabética da chave
 - (e) Leia o texto pela ordem das colunas
3. A saída da segunda rodada é escrita no arquivo de saída

Para descriptografar:

1. Leia o texto cifrado da entrada
2. Por duas rodadas, faça:

- (a) Pegue a chave da rodada na ordem inversa da criptografia
 - (b) Crie a matriz de transposição do texto junto com a chave
 - (c) Complete a última linha da matriz com caracteres
 - (d) Reordene a matriz com base na chave
 - (e) Transponha a matriz
 - (f) Leia a matriz pela ordem das linhas
 - (g) Remova todos os caracteres do final do texto
3. A saída da segunda rodada é escrita no arquivo de saída.

3. Utilização do programa

3.1. AES

Para rodar o AES, utilizamos diretamente a OpenSSL do Linux com a opção 'aes-128-cbc', ou seja, usamos o modo **CBC (Cipher Block Chaining)** para encriptar. Nenhuma outra alteração ou uso foi feito além dos citados.

Exemplo: `openssl aes-128-cbc -e -in entrada -out saida -K chave -iv '00000000000000000000000000000000'`

3.2. Medição de tempo

Para medir o tempo de cada um utilizamos a biblioteca padrão de tempo do Python, 'time' e como função para medir o tempo usamos o `time.perf_counter()`, que ao contrário da função `time.time()` é melhor para o nosso caso por ser bem mais exato em cenários onde se necessita de precisão na medição de tempo.

Basicamente para medir o tempo:

- Salvamos um timestamp com `stamp = time.perf_counter()`
- Rodamos a função desejada
- Após terminar de rodar, calculamos um novo timestamp com `stamp = (time.perf_counter() - stamp)`
- Ao final temos em `stamp` o tempo total tomado pela função

3.3. Flags e execução

Exemplos de uso do programa:

- Criptografar: `./main.py -c Texto.txt -o enc.txt -st`
- Criptografar c/ AES: `./main.py -c Texto.txt -o enc.txt -a`
- Descriptografar: `./main.py -d enc.txt -o dec.txt -st`
- Descriptografar com AES: `./main.py -d enc.txt -o dec.txt -a`

Flags:

- **-c:** Criptografar + Arquivo de entrada
- **-d:** Descriptografar + Arquivo de entrada
- **-o:** Arquivo de saída (**obrigatório**)
- **-s:** Cifra de substituição (pode ser usado junto com -t)
- **-t:** Cifra de transposição (pode ser usado junto com -s)
- **-a:** Cifra AES
- **-b:** Benchmarking (utilizado sozinho para rodar os dois algoritmos e gerar os gráficos de comparação de tempos, bem como os arquivos de saída para comparação)
- **-h:** Mostrar ajuda

Em todas as opções, tanto para criptografar, quanto para descriptografar, será perguntado a(s) chave(s) a ser(em) utilizada(s) e como saída do programa haverá o tempo de execução do algoritmo selecionado.

4. Comparação

4.1. Livros escolhidos

- 1. Lorem Ipsum (10 KB)
- 2. Frankenstein (439 KB)
- 3. Obra Completa de William Shakespeare (5.4 MB)
- 4. Extra - O Chamado de Cthulhu (94 KB)

4.2. Gráficos

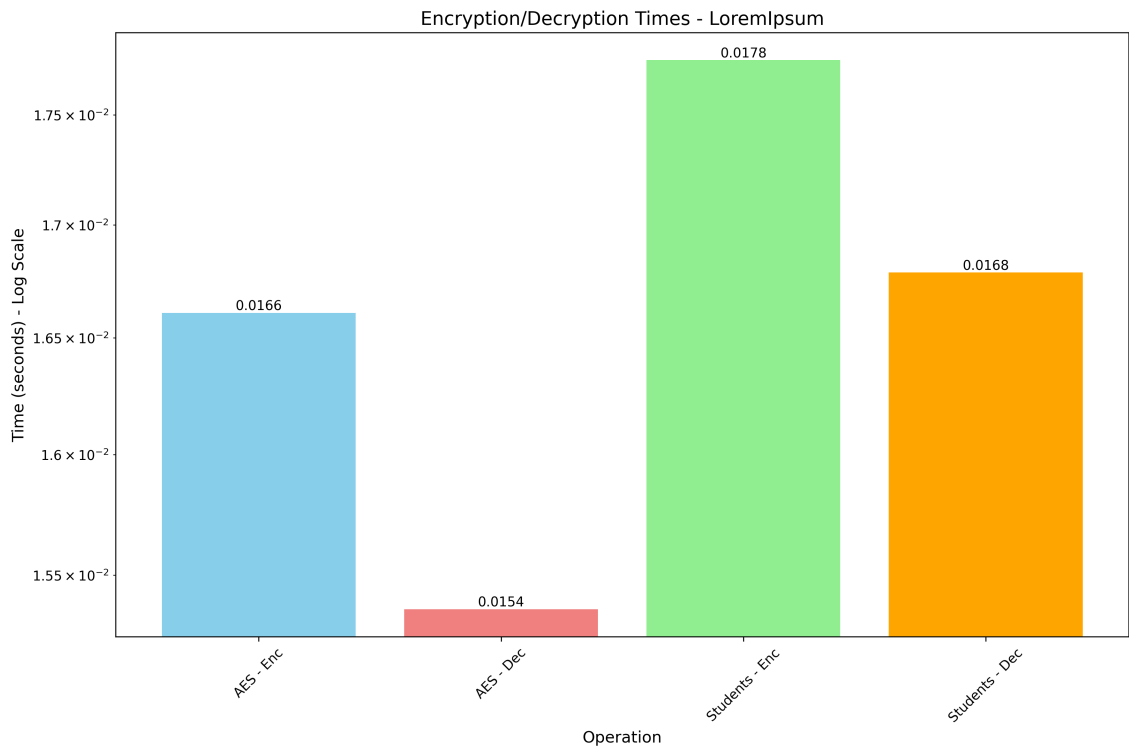


Figura 2. Arquivo de 10 KB (Lorem Ipsum)

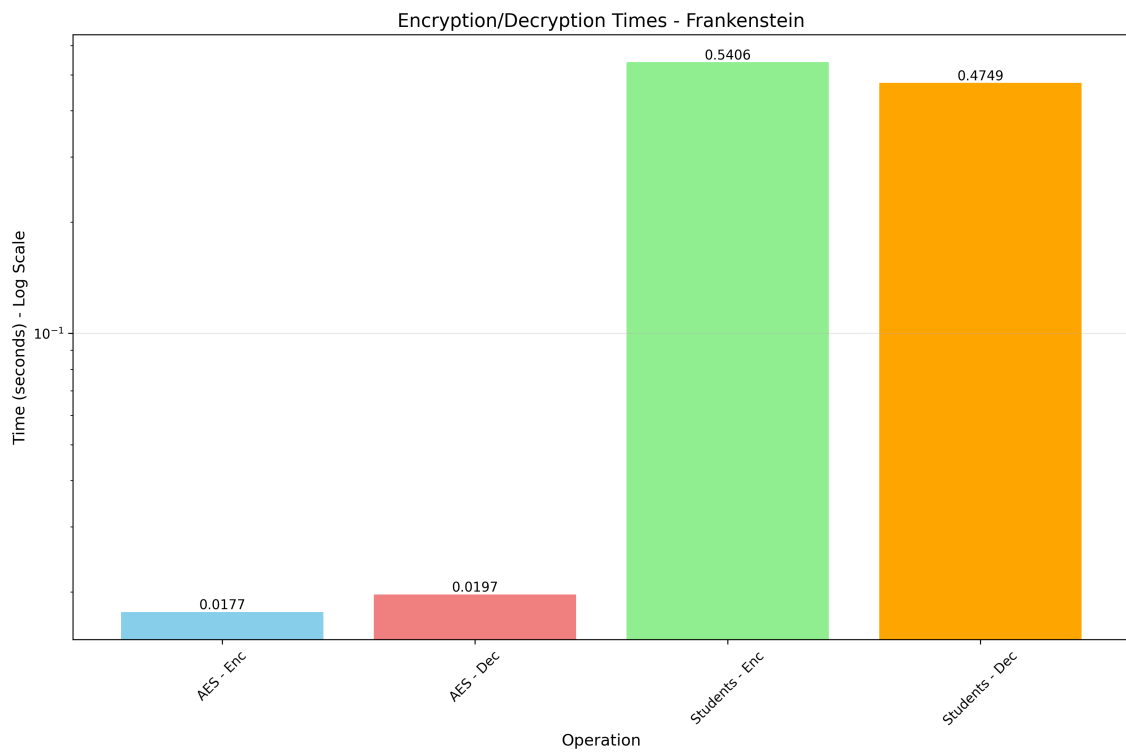


Figura 3. Arquivo de 500 KB (Frankenstein)

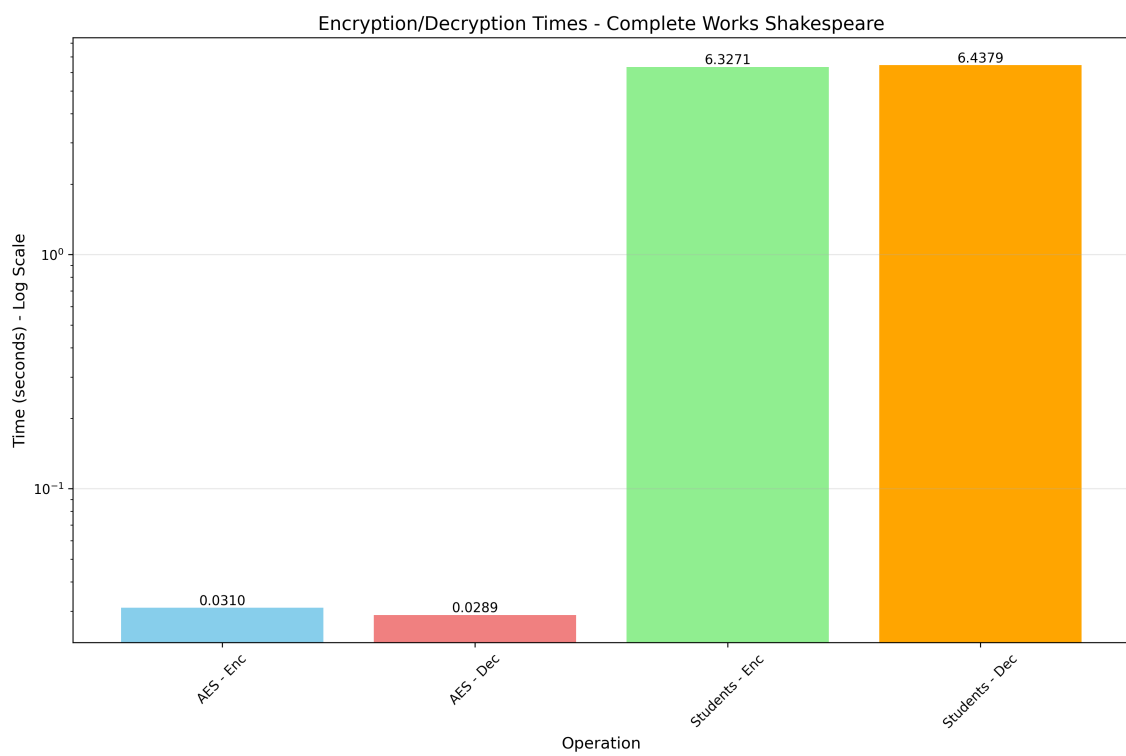


Figura 4. Arquivo de 5 MB (Shakespeare)

4.2.1. Análise dos resultados

Como podemos ver nos gráficos, o AES sempre venceu o nosso algoritmo, às vezes por uma margem menor, como no caso do texto 1. *Lorem Ipsum (10 KB)*, mas nos outros casos por uma margem bem maior, chegando a ser 200x mais lento no caso do texto 3. *Obra Completa de William Shakespeare (5.4 MB)*. Podemos conjecturar que isso ocorre principalmente devido a transposição, que ocorre duas vezes, com duas operações de matrizes em ordem $O(N^2)$ e quatro operações em ordem $O(N)$ em cada ocorrência da transposição, percorrendo um arquivo de $N = 5.638.522$ caracteres para o texto 3.

Na substituição a espera por sorteio de números e conversão de dados também justificam o desempenho menor, assim como o acesso a memória secundária (HD), uma vez que a saída da cifra de substituição é colocada em um arquivo intermediário que será acessado pela cifra de transposição.